

AI ENGINEERING PORTFOLIO PROJECT

ECOS AI Decision Intelligence Platform

Professional Portfolio Project Blueprint

Hybrid machine learning, environmental data retrieval, regulatory RAG, and a bounded AI copilot

TARGET ROLE	AI Engineer Intern / Junior AI Engineer
DOMAIN	Environmental monitoring and B2B contract renewal
STATUS	Portfolio development blueprint August 2026

Synthetic data is disclosed and is not presented as real business impact.

Report Map

This report defines the project purpose, system design, implementation priorities, engineering controls, and recruiter-facing evidence.

1. Executive Summary	14. MLOps and Maintenance
2. Business Problem and Users	15. Testing and Acceptance Criteria
3. Recommended System Architecture	16. Repository Blueprint
4. Functional Scope	17. Delivery Roadmap
5. Renewal Prediction: Problem Definition	18. Scope Priorities
6. Feature and Explanation Design	19. Portfolio Artifacts
7. Synthetic Data Strategy and Cold Start	20. Recommended README Opening
8. ML Training and Evaluation Pipeline	21. CV Entry
9. Environmental Intelligence	22. Interview Narrative
10. RAG Knowledge Service	23. Risks and Mitigations
11. Bounded AI Copilot	24. Success Criteria for Job Search
12. API Contracts	25. Final Recommendation
13. Security and Privacy Design	
INTEGRITY STATEMENT ECOS currently uses synthetic data. Portfolio credibility comes from reproducible data generation, leakage-safe evaluation, deployable services, security, observability, and a clear path to production data.	

1. Executive Summary

ECOS is an environmental monitoring application extended into an AI decision-intelligence platform. It combines a supervised machine-learning service for customer contract-renewal risk, deterministic environmental-data retrieval, retrieval-augmented generation (RAG) for regulations and internal documents, and a bounded AI copilot that explains evidence without inventing measurements or causal claims.

The recommended portfolio scope is intentionally focused: one modular FastAPI backend, one well-defined renewal prediction problem, one RAG knowledge base, and one controlled tool-calling workflow. This demonstrates the capabilities employers expect from an entry-level AI engineer without creating an unfinished multi-agent or microservice system.

Portfolio proposition

Dimension	Demonstrated capability	Recruiter evidence
Data engineering	Time-aware snapshot construction and synthetic event generation	Data dictionary, generator, validation report
Applied ML	Baselines, calibration, temporal validation, leakage prevention	Training pipeline, experiment runs, model card
LLM engineering	Grounded explanation over structured tools and retrieved documents	RAG evaluation, prompt contract, citations
Backend engineering	Typed APIs, validation, error handling, versioned responses	OpenAPI specification and integration tests
MLOps	Model registry, lineage, monitoring, rollback	MLflow runs, monitoring dashboard, runbook
Security	Least privilege, authorization, rate limits, auditability	Threat model and security test checklist
Product thinking	Explicit users, decisions, fallbacks, and acceptance criteria	Demo scenario and architecture decision records

2. Business Problem and Users

Environmental-service companies need to monitor measurements, interpret regulatory obligations, and manage recurring monitoring contracts. ECOS supports two concrete decisions:

1. **Which contracts should customer-success staff review before expiration?**
2. **What verified environmental or regulatory evidence supports an operator's question?**

Primary users

- **Customer-success staff:** prioritize expiring contracts and record follow-up actions.
- **Environmental operators:** retrieve measurements and compare them with configured thresholds.
- **Managers:** view risk distribution, data quality, model health, and audit history.

- **AI/ML engineers:** retrain, evaluate, deploy, monitor, and roll back model versions.

Explicit non-goals for the MVP

- The renewal score is not a causal explanation and does not prove why a customer renews.
- The chatbot must not generate missing pollution measurements.
- Synthetic-data metrics are not claims of real-world accuracy or ROI.
- The system does not autonomously modify contracts, customers, or environmental records.
- Multi-agent orchestration and MCP integration are deferred until a real interoperability need exists.

3. Recommended System Architecture

```

ECOS WinForms Application
  |
  | HTTPS / authenticated local or private-network calls
  v
Modular FastAPI AI Backend
- Query router and bounded orchestrator
- Renewal inference service
- Environmental query service
- RAG retrieval and citation service
- Gemini adapter for language generation
- Audit, metrics, and health endpoints
  |
  +--> SQL Server through a dedicated read-only account
  +--> Versioned ML artifact and MLflow registry
  +--> Qdrant vector store for approved documents
  +--> Gemini API for grounded response generation

```

Why a modular monolith

A single FastAPI deployment keeps the two-day MVP operationally realistic while preserving clean module boundaries. It avoids premature service discovery, distributed tracing, and network failure modes. Modules can be extracted later only when independent scaling, ownership, or release cadence justifies it.

Core request flow

1. The application sends an authenticated request containing user identity and intent.
2. The router selects an allow-listed tool: renewal prediction, measurement retrieval, or document retrieval.
3. The tool returns structured JSON with evidence, provenance, and version metadata.
4. Gemini receives only the minimum approved context and converts it into a concise explanation.
5. The backend validates the response, records an audit event, and returns citations and trace identifiers.

The LLM never directly connects to the database and never invents the ML score or the environmental reading.

4. Functional Scope

Module	Inputs	Outputs	Guardrail
--------	--------	---------	-----------

Module	Inputs	Outputs	Guardrail
Renewal scoring	Customer, contract, prediction date	Probability, risk band, uncertainty, model version	Reject incomplete or post-cutoff features
Environmental query	Site, parameter, time range	Exact readings, units, source timestamps, configured threshold comparison	No fabricated values or unsupported forecasting
Regulatory RAG	User question and approved filters	Grounded answer, cited document sections, retrieval diagnostics	Abstain when evidence is insufficient
AI copilot	Natural-language question	Tool plan, grounded explanation, follow-up suggestion	Allow-listed read-only tools and step limit
Feedback capture	Prediction/answer ID and user rating	Structured feedback event	No silent retraining from unreviewed feedback
Operations	Health, data freshness, drift, latency	Dashboards, alerts, audit trail	Role-based access

5. Renewal Prediction: Problem Definition

Prediction unit

Use one row per **customer-contract prediction snapshot**, not one row per payment/installment period. Several installments from the same contract share the same final renewal outcome and must not be treated as independent labels.

Time semantics

- **Prediction date:** 60 days before the current contract end date.
- **Feature window:** only events with timestamps less than or equal to the prediction date.
- **Outcome window:** 90 days after the current contract expires.
- **Positive label:** a qualifying new contract for the same customer appears within the outcome window.
- **Negative label:** no qualifying renewal appears after the full outcome window has elapsed.
- **Unknown label:** the outcome window has not elapsed; exclude this row from supervised training.

Why the outcome is delayed

Training requires a known result. A snapshot created on March 1 for a contract ending April 30 cannot be labelled negative on March 1 because the customer still has time to renew. The row becomes trainable only after the outcome is observed or the 90-day window closes.

Leakage example

Suppose Contract C-101 ends on April 30 and is renewed on May 20.

- Valid prediction snapshot: March 1.

- Valid features: payments, incidents, contacts, and service events recorded on or before March 1.
- Forbidden features: the May 20 renewal contract, a status updated after March 1, or an installment row retroactively marked `renewed = 1`.

If the future renewal is copied to all historical installment rows, the model can learn an outcome that was unavailable at prediction time. Random train/test splitting can then place near-duplicate rows from the same contract in both sets, producing an unrealistic score such as 99% accuracy.

Suggested analytical tables

`AI\_RenewalSnapshot`

Field group	Example fields
Identity and time	snapshot_id, customer_id, contract_id, prediction_date, contract_end_date
Relationship history	tenure, previous contracts, days since last contact
Commercial behavior	total value, payment delay statistics, outstanding balance
Service behavior	completed rounds, missed appointments, complaint count, SLA incidents
Engagement	contact frequency, recent response rate, unresolved tickets
Data quality	missing-field count, source freshness, snapshot schema version

`ContractRenewalOutcome`

Field	Meaning
contract_id	Contract receiving the prediction
outcome_observed_at	Date when the label became knowable
renewed_contract_id	New qualifying contract, nullable
renewed_label	1, 0, or NULL while pending
label_rule_version	Reproducible business definition

`AI\_PredictionLog`

Stores prediction ID, entity IDs, prediction time, probability, risk band, feature-schema version, model version, request trace, and eventual outcome for monitoring. It must not store sensitive raw prompts unless required and approved.

6. Feature and Explanation Design

Features can be predictive without being causal. Feature importance therefore supports debugging and transparency but must not be presented as a reason that caused renewal.

Recommended feature groups

- Contract recency, frequency, monetary value, and remaining duration.
- Historical renewal count and customer tenure.
- Payment delay rate, balance trend, and late-payment recency.
- Service completion, rescheduling, complaint, and SLA-event statistics.
- Customer engagement and support-interaction recency.
- Data-quality indicators and missingness flags.

Feature acceptance tests

Each feature must have an owner, definition, timestamp, valid range, null policy, and offline/online parity test. Remove a feature when it is unavailable at inference time, duplicates the label, is unstable without governance, or has no defensible business meaning.

Safe explanation contract

The ML service returns structured evidence, not free-form causal prose:

```
{
  "prediction_id": "pred_2026_000184",
  "renewal_probability": 0.63,
  "risk_band": "medium",
  "top_signal_contributions": [
    {"feature": "payment_delay_rate_90d", "direction": "decreases_score", "magnitude": 0.11},
    {"feature": "completed_service_ratio", "direction": "increases_score", "magnitude": 0.08}
  ],
  "model_version": "renewal-xgb-0.3.0",
  "feature_schema_version": "renewal-v2",
  "limitations": ["Predictive associations are not causal explanations."]
}
```

Gemini may translate these fields into user-friendly language, but the API contract and prompt must require terms such as “associated signal” rather than “cause.”

7. Synthetic Data Strategy and Cold Start

Synthetic data is acceptable for system development when clearly disclosed. It is useful for validating schemas, temporal pipelines, security, deployment, monitoring, and failure handling. It is not evidence that the model generalizes to a real population.

Event-based generator

Generate customers, contracts, installments, payments, service rounds, complaints, contacts, incidents, and renewal events on a timeline. Add configurable:

- class imbalance and seasonality;
- correlated behaviors without directly encoding the label;
- missing values, duplicates, delayed events, and invalid records;
- concept drift between time periods;
- cold-start customers and previously unseen categories;

- outcome delay and censored labels.

Version the generator configuration, random seed, schema, and generated dataset. Include tests that fail when post-prediction events enter features.

Real-company cold-start plan

1. Begin with transparent heuristic prioritization and human review.
2. Define event instrumentation, data contracts, and the future label before data collection.
3. Log snapshots and outcomes without claiming model automation.
4. Train a simple baseline only after enough mature outcomes exist.
5. Run the model in shadow mode and compare it with the heuristic.
6. Calibrate thresholds with business costs and deploy with an override path.
7. Monitor drift, calibration, and subgroup behavior; retrain only under a reviewed policy.

The portfolio demo should implement both the heuristic fallback and the ML path. This proves understanding of the transition from no data to operational learning.

8. ML Training and Evaluation Pipeline

Training stages

1. Validate source schemas and business rules.
2. Build point-in-time snapshots using the prediction cutoff.
3. Exclude pending outcomes and leakage-prone columns.
4. Split data by time; group related rows by customer when overlap is possible.
5. Train baselines: dummy classifier and logistic regression.
6. Compare tree models: random forest and one gradient-boosting implementation.
7. Tune only on the training/validation periods.
8. Calibrate the selected probability model on untouched validation data.
9. Evaluate once on the latest holdout period.
10. Register artifacts, schema, metrics, code revision, and dataset version.

Validation design

Train:	oldest mature snapshots
Validation:	subsequent mature snapshots
Test:	latest fully observed snapshots
Production:	current snapshots with outcomes still pending

Time-based splitting matches deployment: learn from the past and predict a later cohort. Group isolation prevents the same customer's highly similar records from appearing on both sides of a split.

Required metrics

Category	Metrics	Purpose
----------	---------	---------

Category	Metrics	Purpose
Ranking	ROC-AUC, PR-AUC	Compare ordering quality; PR-AUC is important for imbalance
Classification	precision, recall, F1, confusion matrix	Evaluate a chosen operational threshold
Probability	log loss, Brier score, calibration curve	Verify that predicted probabilities are meaningful
Business simulation	recall at review capacity, expected cost/benefit	Connect threshold choice to workflow
Reliability	latency, error rate, missing-feature rate	Validate serving behavior

Accuracy alone is insufficient. A model predicting the majority class can appear accurate while failing to identify contracts requiring attention.

Model acceptance gate

Promote a model only when it beats the heuristic and simple baseline on the latest holdout, passes leakage and schema tests, stays within latency limits, has acceptable calibration, and has a documented rollback target. With synthetic data, describe this as a pipeline acceptance result, not a production business result.

9. Environmental Intelligence

The database is the authority for measurements. The environmental service returns exact values, units, timestamps, station/site identity, quality flags, and data provenance.

Threshold comparisons must reference a versioned rule source and state whether the result is a configured system comparison or a legal conclusion. When measurements are absent, the assistant must say that no verified data is available.

Recommended response fields include `measurement_id`, `parameter`, `value`, `unit`, `measured_at`, `source_system`, `quality_flag`, `threshold_rule_id`, and `threshold_comparison`.

10. RAG Knowledge Service

RAG is appropriate for regulations, internal procedures, monitoring-method guides, and customer-facing service documentation. It is not a substitute for structured SQL retrieval.

Ingestion pipeline

1. Approve and inventory source documents.
2. Extract text while preserving document and section metadata.
3. Normalize encoding and remove repeated headers/footers.
4. Chunk by semantic section with controlled overlap.
5. Embed and index chunks in Qdrant.
6. Store source title, version, effective date, page/section, access scope, and checksum.

7. Evaluate retrieval before enabling generation.

Runtime pipeline

Apply authorization filters, retrieve candidates, optionally rerank, enforce a relevance threshold, generate only from retrieved evidence, and return citations. If evidence is weak or conflicting, abstain and request a more specific question.

RAG evaluation set

Create 30-50 reviewed questions containing answerable questions, unanswerable questions, conflicting-version cases, and access-controlled cases. Track retrieval recall@k, mean reciprocal rank, citation correctness, groundedness, abstention accuracy, latency, and cost.

11. Bounded AI Copilot

The portfolio should implement one tool-calling copilot rather than a multi-agent system. The copilot may call only:

- `get_renewal_prediction(customer_id, contract_id);`
- `get_environmental_measurements(site_id, parameter, from, to);`
- `search_approved_documents(query, filters);`
- `get_prediction_history(prediction_id).`

Control policy

- Read-only tools for the MVP.
- Maximum tool-call and reasoning-step limits.
- Typed arguments and output validation.
- Per-tool authorization checks.
- Timeout, retry, and circuit-breaker policies.
- Audit log with user, selected tool, latency, status, and trace ID.
- No automatic database update or contract action.

MCP is optional later when ECOS must expose or consume tools across multiple compatible clients. Adding MCP solely as a trend keyword increases the attack surface without strengthening the core demo.

12. API Contracts

Renewal endpoint

POST /v1/renewal/predict

Input: customer ID, contract ID, optional valid prediction date.

Output: prediction ID, calibrated probability, risk band, structured signal contributions, model and schema versions, data timestamp, and limitations.

Copilot endpoint

POST /v1/copilot/query

Input: user query and conversation identifier; authorization identity comes from the token, not the prompt.

Output: answer, evidence objects, citations, tool trace summary, model versions, request ID, and safety flags.

Operational endpoints

- GET /health/live for process liveness.
- GET /health/ready for database, model, vector-store, and configuration readiness.
- GET /metrics for protected operational metrics.
- GET /v1/models/current for approved model metadata.

Use Pydantic validation, stable error codes, request correlation IDs, structured logging, and versioned API routes.

13. Security and Privacy Design

An API does not have to be reachable from the public Internet. ECOS can bind the AI backend to 127.0.0.1 for same-machine use or to a private LAN/VPN address for controlled internal access. If remote public access is required, place it behind a managed gateway or reverse proxy with TLS and authentication.

Required controls

Layer	Control
Network	Local/private binding by default; firewall allow-list; TLS at ingress
Identity	Short-lived tokens or session-bound authentication; no identity from prompt text
Authorization	Role and object-level checks for every customer/site request
Database	Dedicated read-only account, parameterized queries, stored procedures or reviewed query layer
Secrets	Environment/secret store; never source control or logs
Abuse	Rate limits, body-size limits, timeouts, retry budgets, quotas
LLM	Prompt-injection isolation, retrieved-content labeling, tool allow-list, output validation
Privacy	Data minimization, PII redaction, retention policy, encrypted transport
Supply chain	Exact dependency versions, artifact checksum/signature, vulnerability scanning
Audit	Immutable request, tool, model-version, authorization, and outcome events

The ML API is not safe merely because it reads from a database. SQL injection, broken object-level authorization, stolen credentials, model deserialization, denial of service, and sensitive-data leakage remain relevant threats.

14. MLOps and Maintenance

Versioned assets

- Raw and processed dataset snapshots.
- Synthetic generator configuration and random seed.
- Label-rule and feature-schema versions.
- Training code revision and environment lock file.
- Model artifact, checksum, metrics, and decision threshold.
- Prompt template, LLM configuration, embedding model, and RAG index.

Lifecycle

1. A scheduled or manually approved pipeline validates new mature outcomes.
2. A challenger model is trained and evaluated against the champion.
3. Automated gates produce a report; a human approves promotion.
4. The new version runs in shadow or canary mode.
5. Monitoring compares performance, calibration, latency, and data quality.
6. Rollback restores the previous model, threshold, prompt, or index version.

Monitoring

- Input drift, missingness, category changes, and feature freshness.
- Prediction distribution, uncertainty, calibration once labels mature, and subgroup slices.
- API latency, saturation, error rate, dependency health, and retry counts.
- Retrieval relevance, groundedness, citation accuracy, abstention, LLM latency, and token cost.
- Security events such as denied object access, rate-limit triggers, and prompt-injection detections.

Retraining should be triggered by an approved schedule or evidence such as enough new mature outcomes and material drift. Automatic retraining on every new row is unsafe and unnecessary.

15. Testing and Acceptance Criteria

Test layers

- **Unit tests:** label rules, cutoff filters, feature transformations, risk bands, prompt builders.
- **Data tests:** schema, ranges, uniqueness, referential integrity, timestamp consistency, label maturity.
- **Leakage tests:** forbid future timestamps, outcome-derived fields, and customer overlap across inappropriate splits.
- **Model tests:** baseline comparison, calibration, deterministic seed, artifact load, schema compatibility.

- **API tests:** validation, authentication, authorization, timeouts, duplicated-submit prevention, error mapping.
- **RAG tests:** retrieval, citations, unanswerable cases, conflicting versions, injection attempts.
- **End-to-end tests:** WinForms request through tool execution to evidence-backed display.
- **Load and resilience tests:** concurrency, dependency outage, quota exhaustion, fallback behavior.

MVP acceptance criteria

- A contract prediction is reproducible from its snapshot and model version.
- No feature timestamp is later than the prediction date.
- Pending outcomes never enter supervised training.
- The API returns one response for one UI action and protects against duplicate submission.
- Environmental answers contain exact provenance or an explicit no-data response.
- RAG answers include valid citations or abstain.
- Gemini failure produces a structured fallback rather than blocking the core ML result.
- Secrets are absent from source control and logs.
- A previous model version can be restored using the runbook.

16. Repository Blueprint

```
ecos/
|-- app/                                # Existing WinForms application
|-- ai-service/
|   |-- src/
|   |   |-- api/                        # FastAPI routes and schemas
|   |   |-- renewal/                   # Features, inference, explanations
|   |   |-- environment/               # Structured measurement queries
|   |   |-- rag/                       # Ingestion, retrieval, citations
|   |   |-- copilot/                   # Router and bounded tool policy
|   |   |-- security/                  # Auth, authorization, redaction
|   |   |-- observability/             # Logs, metrics, tracing
|   |-- tests/
|   |-- Dockerfile
|-- ml/
|   |-- data_generation/
|   |-- feature_pipeline/
|   |-- training/
|   |-- evaluation/
|   |-- model_card/
|-- knowledge-base/
|   |-- approved_sources/
|   |-- evaluation/
|-- infra/                              # Compose, configuration examples
|-- docs/
|   |-- architecture/
|   |-- threat-model/
|   |-- runbooks/
|   |-- portfolio/
|-- .github/workflows/
|-- README.md
|-- LICENSE
```

The separate experimental folder `ai-taiky-api` can be migrated into `ml/` and `ai-service/src/renewal/` after its data and model code are audited. Do not copy an opaque serialized model into the application repository without its training lineage and dependency lock.

17. Delivery Roadmap

First 48 hours: recruiter-ready MVP

Timebox	Deliverable	Evidence
Hours 0-4	Freeze problem, label, prediction date, architecture	ADR and task board
Hours 4-10	Build temporal synthetic generator and snapshot dataset	Data dictionary and validation output
Hours 10-16	Train baselines and one tree model with temporal split	MLflow comparison and model card draft
Hours 16-22	Package renewal inference in FastAPI	OpenAPI and API tests
Hours 22-28	Add read-only environmental retrieval	Provenance response and authorization test
Hours 28-34	Index a small approved document set in Qdrant	Retrieval evaluation and cited answer
Hours 34-39	Add Gemini explanation and bounded routing	Trace showing structured tool use
Hours 39-44	Add Docker Compose, health checks, logs, and security baseline	One-command startup and threat checklist
Hours 44-48	Record demo, finish README, diagrams, CV bullets	Public portfolio package

Additional 3-5 days: engineering hardening

- Champion/challenger workflow and rollback drill.
- Drift and delayed-outcome monitoring dashboard.
- Expanded RAG evaluation set and prompt-injection tests.
- CI pipeline for linting, unit tests, API tests, and container build.
- Load testing and dependency-failure simulation.
- Optional local Qwen experiment with measured hardware, latency, and quality trade-offs.

18. Scope Priorities

Must have

- Leakage-safe renewal dataset and transparent synthetic-data statement.
- Baseline-to-model comparison with probability calibration.
- Versioned FastAPI prediction endpoint.
- Deterministic environmental-data retrieval.
- Small evaluated RAG knowledge base with citations.

- Gemini used only for grounded explanation.
- Security baseline, logs, health checks, Docker setup, tests, README, model card, and demo.

Should have

- MLflow experiment tracking and registry.
- Qdrant metadata filters and retrieval evaluation.
- Heuristic fallback and LLM-unavailable fallback.
- CI workflow and monitoring dashboard screenshot.

Defer

- Multi-agent architecture.
- MCP server without a concrete external client.
- Fine-tuning an LLM.
- Kubernetes and premature microservices.
- Automated database writes by the copilot.
- Claims of production accuracy based on synthetic data.

19. Portfolio Artifacts

The repository should contain:

1. A README with problem, users, architecture, quick start, demo flow, limitations, and results.
2. A system-architecture diagram and one sequence diagram.
3. A data dictionary and label-definition document.
4. A model card including limitations, metrics, calibration, and intended use.
5. An experiment comparison exported from MLflow.
6. A RAG evaluation report with grounded and abstention examples.
7. An API specification with example requests and responses.
8. A concise threat model and security checklist.
9. A monitoring/rollback runbook.
10. A two-to-four-minute demo video or GIF.

20. Recommended README Opening

ECOS is a production-oriented AI decision-intelligence extension for an environmental monitoring system. It combines leakage-safe contract-renewal scoring, deterministic environmental-data retrieval, citation-backed regulatory RAG, and a bounded AI copilot behind a versioned FastAPI service. The current dataset is synthetic and is used to validate the engineering lifecycle, not to claim real-world business accuracy.

21. CV Entry

ECOS AI Decision Intelligence Platform — Personal Project

- Designed a modular FastAPI AI backend that combines calibrated contract-renewal scoring, SQL-based environmental retrieval, regulatory RAG, and Gemini-generated grounded explanations.
- Built a time-aware synthetic event pipeline with explicit prediction cutoffs, delayed outcomes, temporal validation, leakage tests, and reproducible dataset/model versioning.
- Implemented typed API contracts, read-only tool calling, authorization boundaries, audit logging, health checks, Docker-based deployment, fallback behavior, and model rollback documentation.
- Evaluated ML ranking and calibration alongside RAG retrieval, citation, groundedness, and abstention quality; clearly separated synthetic pipeline results from production claims.

Do not place invented percentage improvements in the CV. Replace these bullets with measured repository facts only after the corresponding artifacts and tests exist.

22. Interview Narrative

Use this concise story:

1. ECOS began as an environmental CRUD application and an isolated renewal-model experiment.
2. The initial synthetic model score was unreliable because label timing, duplicated contract periods, and random splitting could leak future outcomes.
3. The redesign established a customer-contract prediction snapshot, delayed label maturity, temporal evaluation, and an honest synthetic-data boundary.
4. Structured tools became the source of facts; RAG supplied approved document evidence; Gemini only explained retrieved or predicted evidence.
5. The result demonstrates the full AI engineering lifecycle: problem framing, data, ML, serving, LLM grounding, security, tests, monitoring, and rollback.

23. Risks and Mitigations

Risk	Consequence	Mitigation
Synthetic model appears unrealistically strong	Recruiter distrust	Disclose generator assumptions; report pipeline validation, not business lift
Outcome or future event enters features	Inflated offline metrics	Point-in-time joins, blocked columns, temporal tests
Gemini fabricates a measurement or reason	Unsafe answer	Structured evidence, strict prompt contract, validation, abstention
Unauthorized customer/site lookup	Privacy breach	Token identity, object-level authorization, audit logs
Gemini quota or network outage	Chat failure	Return ML/SQL result directly with deterministic explanation template

Risk	Consequence	Mitigation
Serialized model is tampered with	Code execution or wrong predictions	Trusted artifacts, checksums, restricted registry, exact dependencies
RAG uses an obsolete regulation	Incorrect guidance	Effective-date metadata, version filters, source approval workflow
Project scope exceeds two days	Incomplete demo	Must/should/defer priorities and one bounded copilot

24. Success Criteria for Job Search

The project succeeds when a reviewer can clone it, start it with documented commands, execute a repeatable demo, inspect tests and experiment evidence, understand all synthetic-data limitations, and discuss concrete trade-offs with the candidate. A smaller system meeting these criteria is stronger than a larger system containing untested agent or fine-tuning claims.

25. Final Recommendation

Build ECOS as a **hybrid decision-intelligence system**, not as a generic chatbot and not as a model-only demo. The renewal model should make a narrow, auditable prediction; SQL should remain the authority for measurements; RAG should ground document knowledge; and Gemini should provide the language interface. Prioritize temporal data correctness, calibration, API contracts, security, evaluation, deployment, and monitoring. These choices create a credible AI Engineering portfolio even without proprietary data.

References and Market Alignment

- Innotech Vietnam, AI Engineer Intern role emphasizing RAG, Docker, and API experience: <https://vn.linkedin.com/jobs/view/ai-engineer-intern-rag-docker-api-at-innotech-vietnam-corporation-4366457226>
- FPT Software, Student and New Graduate recruitment program: <https://fptjobs.com/SVCNTS2026>
- GreenNode, AI Intern opportunity: <https://career.vng.com.vn/tim-kiem-viec-lam/chi-tiet/6573-ai-intern-greennode-vi>
- OWASP API Security Top 10 (2023): <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- scikit-learn, model persistence: https://scikit-learn.org/stable/model_persistence.html
- scikit-learn, probability calibration: <https://scikit-learn.org/stable/modules/calibration.html>
- MLflow Tracking documentation: <https://mlflow.org/docs/latest/ml/tracking/>
- Model Context Protocol security best practices: https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
- llama.cpp OpenAI-compatible server documentation: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>

Document owner: ECOS project maintainer

Recommended review cadence: update after each milestone or material architecture decision

Next action: implement the 48-hour MVP scope and replace blueprint statements with measured repository evidence.

END OF REPORT
